FOCEX agent: from a POSCAR to phonons

`FOCEX/AGENT/focex-agent.py` – user manual

ALATDYN

October 2026

Contents

1 What it does **1**

2 Requirements **1**

3 Running it **2**

4 Defaults **3**

5 Existing data (FOCEX only) **3**

6 Supercell k-mesh **4**

7 Project folder **4**

8 Thermal conductivity **4**

9 Overriding any file **4**

10 Born charges **5**

11 Testing without VASP **5**

1 What it does

`FOCEX/AGENT/focex-agent.py` runs the FOCEX workflow in one of two ways:

- **full** (`-s POSCAR`): from the primitive cell of a crystal – VASP relaxation and SCF, Born charges (DFPT), SC-SNAPS snapshots, the supercell VASP runs, the force collection, the five `*.params` files, FOCEX and the plots;
- **FOCEX only** (`DATA: <folder>` in the instructions): from existing `POSCARi`/`FORCEDISPi` (and optionally Born charges) – the `*.params` files, FOCEX and the plots (Section 5).

Every step uses defaults that can be overridden, and every step is checked and written to `report.log`. The folder `FOCEX/AGENT` holds the script, this manual, an example input (`focex-agent.example.txt`) and an end-to-end test without VASP (`tests/`). `FOCEX/focex-agent.py` forwards to `AGENT/focex-agent.py`, so run scripts of older projects keep working.

2 Requirements

- VASP-Flow (https://github.com/KeivanS/VASP-Flow) in `~/VASP-Flow` (or `$VASP_FLOW_DIR`): the agent imports its INCAR/KPOINTS generator, GGA+U and MAGMOM rules, KPAR/NCORE rule, POTCAR builder, profiles and high-symmetry k-path. The POTCAR library is `$VASP_POTCAR_DIR`; the VASP binary `$VASP_STD` (or the profile).
- Python with numpy, spglib and pymatgen (VASP-Flow's environment, e.g. `~/vaspenv/bin/python`).
- `sc_snaps.x` (SC-SNAPS), `read_outcar.x` (`FOCEX/utility`), the FOCEX binary, and gnuplot for the plots.

The agent sets `OMP_NUM_THREADS=1` for the VASP it starts: an OpenMP-threaded BLAS inside each MPI rank otherwise starts one thread per core in every rank (a 96-atom cell made no SCF step in 30 minutes on 8 ranks).

3 Running it

```
cp FOCEX/AGENT/focex-agent.example.txt instructions.txt # Project and RANKS are required
~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py init -i instructions.txt -s POSCAR --profile
     rivanna
cd <Project> && sbatch run_workflow.sh # bash run_workflow.sh on a workstation

# FOCEX only, on existing POSCAR1/FORCEDISP1: DATA: <folder> in instructions.txt
~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py init -i instructions.txt
~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py run <Project>

~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py run <Project> --only snaps # redo one task
~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py run <Project> --retry-failed # =
    resubmit_failed.sh
~/vaspenv/bin/python FOCEX/AGENT/focex-agent.py plots <Project> # remake the plots
```

`run_workflow.sh` calls `focex-agent.py run <Project>`, which does the tasks below in order. A finished task is recorded in `state.json` and skipped, so after a time limit the same script is simply submitted again.

task	what it does
`relax`	Niggli-reduced, oriented cell (`POSCAR.init`); VASP relaxation in `01_relax`, restarted from CONTCAR up to three times if EDIFFG is not reached
`poscar0`	CONTCAR Niggli-reduced and oriented again: `POSCAR0`, the cell and Cartesian frame of every later step
`scf`	`02_scf`: same INCAR (ISIF = 1, NSW = 0) and KPOINTS; reuses CHGCAR/WAVECAR when the basis is unchanged; band gap from EIGENVAL

`dfpt`	`02_scf/dfpt`, only if there is a gap (`DFPT: auto`; `on`/`off`): ε_∞ and Born charges
`snaps`	`03_snaps`: `cell.inp`, `supercell.inp`, `snaps.inp`; SC-SNAPS; if the rms displacement is not within 15 % of `DISP_TARGET` the temperature is rescaled and SC-SNAPS rerun (at most twice); the undistorted supercell is checked to be commensurate with POSCAR0
`supercells`	`04_supercells/snap_NNN`: INCAR as the SCF plus ISYM = 0, Gamma mesh (Section 6), MAGMOM expanded to the supercell
`vasp_snaps`	the supercell runs, one after the other; failures are diagnosed in `report.log` and `resubmit_failed.sh` reruns them (an OUTCAR is never overwritten)
`collect`	`read_outcar.x` in every folder: `FORCEDISP1`; `POSCAR1` = `poscar_000`; checks the net force and that the OUTCAR positions are those of the POSCAR
`focex`	`05_focex`: the five `*.params` files and FOCEX; with Born charges also a `NOBORN` run; if equations/unknowns < `LASSO_RATIO` (2) also a `LASSO` run; the DOS range is reset to 1.1× the highest frequency when needed. A FOCEX run counts as failed if its exit code is not 0
`plots`	`bvel.plt`, `all.plt`, `qha.plt` with their placeholders filled and `makeplots.sh`, in every FOCEX folder

For each FOCEX run the report gives the fit error, the numbers of equations and unknowns, the lowest ω^2 (a warning below $-25\,\mathrm{cm}^{-2}$) and the highest frequency (a warning above $2500\,\mathrm{cm}^{-1}$).

4 Defaults

- `ACCURACY: Accurate` – ENCUT 600 eV, EDIFF 10^{-7}, EDIFFG -0.001, PREC = Accurate in every step; `Normal` – 450 eV, 10^{-6}, -0.003, PREC = Normal (cheaper, but the undistorted 96-atom ZnO supercell then kept forces of $\sim$0.01 eV/Å). DFPT uses EDIFF 10^{-8}.
- Relaxation and SCF k-mesh: VASP-Flow `coarse` (1000 k-points per reciprocal atom), `fine` (6000) if every element is a metal.
- `SUPERCELL: SMALL` – the multiple of the conventional cell with the fewest atoms whose smallest face distance is at least 10 Å, within 100 atoms (`LARGE`: 14 Å, 300 atoms).
- 50 snapshots (`SNAPSHOTS`), 30 K, $400\,\mathrm{cm}^{-1}$, target rms displacement 0.02 Å. The number of equations is $3N_{\mathrm{atoms}}$ per snapshot; 20 snapshots of 96 atoms give 5760 equations, against 264 unknowns for ZnO with ranks 1–3.
- `latdyn.params` mesh `fine`; `kpbs.params` from the automatic path.
- `RANKS` has no default; `RANGE: 6 2 1 0 0 0 0` shells for ranks 2–8; `FC2RANGE: 0`.

`structure.params`: line 1 $a\,b\,c\,\alpha\,\beta\,\gamma$ of the conventional cell, line 2 the primitive vectors in conventional units, line 3 `1.0 0`. POSCAR0 is rotated so that its conventional cell lies exactly where FOCEX and SC-SNAPS build it (a along x, b in the xy plane), so POSCAR1, FORCEDISP1 and the Born charges are all in that frame.

5 Existing data (FOCEX only)

Give `DATA: <folder>` in the instructions (no `-s`). The folder holds `POSCAR1`, `FORCEDISP1` (and `POSCAR2`, `FORCEDISP2`, ...; all pairs are used) in any orientation, and optionally

- `POSCAR`: the primitive cell in the frame of the data; otherwise spglib finds it from `POSCAR1`, keeping its orientation;
- `dielectric.params`: ε_∞ and the Born charges in the frame of the data, one block per atom of that `POSCAR` (without a `POSCAR` only with one atom per species).

`SPECIES: Mg O` names the species when the POSCARs have no element line. `init` symmetrizes the lattice of the primitive cell (spglib's idealized lattice parameters, same basis and atom order: lattice parameters given to 6–7 digits make FOCEX's symmetry analysis lose force-constant groups), rotates it into the FOCEX frame, rotates `POSCARi` and the positions and forces of `FORCEDISPi` with the same matrix and ε and Z as $R^T T R$, writes `POSCAR0` and `04_supercells/`, and marks relax ... collect as provided; `run` then does `focex` and `plots`. The report gives the rotation and the size of the symmetrization.

Checked on MgO data in a rhombohedral orientation: without Born charges the fit error (7.197 %), the 35 unknowns, the highest frequency (647 cm^{-1}), the bulk and shear moduli equal those of a FOCEX run in the original frame.

6 Supercell k-mesh

If the supercell matrix (rows, primitive units) is diagonal up to a permutation, the mesh is the primitive one divided by the multiples, rounded up (6 10 8 with 2×2×2 gives 3 5 4). Otherwise – every multiple of a centred conventional cell – the supercell mesh keeps the k-point spacing of the primitive mesh and is in general not exactly commensurate. `KPOINTS_snaps` sets it by hand.

7 Project folder

`instructions.txt`	copy of the input (`init` appends the k-mesh density it chose)
`POSCAR.input, POSCAR.init`	the structure given; reduced and oriented
`POSCAR0`	relaxed cell, frame of every later step
`state.json`	finished tasks, supercell matrix, gap, ε, Born charges
`report.log`	one entry per task: OK/FAILED, minutes, what was checked
`run_workflow.sh`	the batch script (VASP-Flow profile); `resubmit_failed.sh`
`01_relax, 02_scf`	VASP; `02_scf/dfpt` the DFPT run
`03_snaps`	SC-SNAPS input and `poscar_NNN`
`04_supercells`	`snap_NNN/` one VASP run each; `FORCEDISP1`, `POSCAR1`
`05_focex`	the FOCEX run and plots; `NOBORN/`, `LASSO/` when made

WAVECAR, CHG, CHGCAR, WFULL, TMPCAR and PROCAR are deleted after use; OUTCARs never.

8 Thermal conductivity

THERMACOND reads `lat_fc.dat`, `fc2.dat` and `fc3.dat` (and, if present, `structure.params`: since October 2026 the cell is also taken from `lat_fc.dat`); give at least `RANKS: 1 1 1 0 0 0 0 0`. The agent does not run it: set lines 8–13 of `latdyn.params` in `05_focex` and run `kap8` there.

9 Overriding any file

A block named after a file replaces it:

```
structure_params
... the whole file ...
structure_params_END
```

`INCAR` (every step) and `INCAR_relax`, `INCAR_scf`, `INCAR_dfpt`, `INCAR_snaps` are merged tag by tag into the generated INCARs. The other blocks are written verbatim: `KPOINTS`, `KPOINTS_snaps`, `cell_inp`, `supercell_inp`, `snaps_inp`, `structure_params`, `latdyn_params`, `kpbs_params` (required with `KPATH: 1`), `dielectric_params`, `default_params`. `END_<NAME>` is also accepted, in any case.

10 Born charges

VASP prints Z^*_{ij} with i the field and j the displacement direction. FOCEX contracts the wave vector with the second index of `charge(i,j)` (`aux_na.f90`), so the agent writes the transpose; `BORN_TRANSPOSE: no` writes them as printed.

11 Testing without VASP

`tests/run_test.sh` runs the full workflow on fcc argon with `tests/fake_vasp.py` (a nearest-neighbour Lennard-Jones crystal) in place of VASP and compares FOCEX's phonons with the exact ones (`tests/exact_phonons.py`): within $0.23\,\mathrm{cm}^{-1}$ (0.4 %), elastic constants within 0.8 %.